

Estudo Dirigido: Arquitetura ARM Cortex-M

Esta atividade tem por finalidade consolidar o conteúdo do capítulo Introdução aos Processadores ARM Cortex-M, propor uma investigação dirigida no livro-texto da disciplina e, por fim, encerrar-se com uma pergunta que antecipa o assunto do capítulo Os Microcontroladores STM32 e a Série F4, dedicado ao microcontrolador STM32F411.

i Orientações

- **Respostas autorais:** elabore suas próprias explicações a partir daquilo que foi estudado e pesquisado. Respostas copiadas integralmente de fontes externas, ou geradas por inteligência artificial sem elaboração própria, não atingem o objetivo pedagógico da atividade.
- **Fontes:** nas questões que envolvem pesquisa externa, indique sempre, e de forma explícita, as referências consultadas (links, manuais do fabricante, livros, entre outras).
- **Livro-texto:** as questões pertencentes à seção “Investigação no Livro-Texto” exigem a consulta ao livro de Joseph Yiu, *The Definitive Guide to ARM® Cortex®-M3 and Cortex®-M4 Processors*, 3ª edição (Newnes, 2013).
- **Entrega:** redija as respostas em um arquivo `ESTUDO-DIRIGIDO-02.md`, na raiz da sua pasta `~/sembl-workspace`, e envie ao seu repositório de entregas com `git push` (o repositório é configurado no roteiro de Ambiente e Ferramentas de Desenvolvimento).

Arquitetura e Modelo de Programação

1. Explique por que razão o Cortex-M4 é classificado como uma arquitetura **Harvard modificada**. Quais são os três barramentos do núcleo (I-Code, D-Code e System), e de que maneira essa divisão permite que o processador busque a próxima instrução na Flash ao mesmo tempo em que a instrução atual acessa um dado? Em que sentido, ainda assim, ele se comporta como Von Neumann do ponto de vista de quem programa?
2. Descreva as combinações possíveis entre os **modos de operação** (*Thread Mode* e *Handler Mode*) e os **níveis de privilégio** (privilegiado e não-privilegiado). Por que razão o *Handler Mode* é, invariavelmente, privilegiado?
3. O Cortex-M possui dois ponteiros de pilha, o **MSP** e o **PSP**, sendo que apenas um deles corresponde ao SP efetivamente em uso a cada instante. Qual dos dois é utilizado imediatamente após o reset e durante as exceções? Por que motivo um sistema operacional de tempo real se beneficia de manter uma pilha separada para as tarefas (PSP) e outra distinta para o *kernel* e as exceções (MSP)?
4. O que diferencia, de fato, o conjunto **T32** (Thumb-2) do **T16** (Thumb original)? De que forma o processador executa instruções de 16 e de 32 bits intercaladas, sem que seja necessário alternar entre estados de execução? Cite um exemplo, observado na prática, de uma instrução que exigiu codificação de 32 bits, e justifique o motivo.
5. Considere a expressão em C `data = *ptr++;`, na qual `ptr` aponta para um vetor de inteiros de 32 bits. Qual é a instrução ARM correspondente, e qual modo de endereçamento ela emprega? Diferencie esse modo daquele pré-indexado (`[Rn, #offset]!`) e do deslocamento imediato sem atualização (`[Rn, #offset]`).

6. Por que razão a faixa de endereços dos periféricos (`0x40000000` a `0x5FFFFFFF`) é classificada, pela arquitetura, como memória **Device**, e não como memória *Normal*? Quais garantias a arquitetura oferece para os acessos realizados nessa região (quanto à especulação, à ordem, ao número de acessos e à execução de código)? Qual seria o risco decorrente de uma leitura especulativa em um registrador de estado de um periférico de comunicação? Sendo a memória *Device bufferable*, em qual situação o programador necessita empregar uma instrução **DSB**?

Investigação no Livro-Texto

7. Consulte a Seção 4.3.1 do livro do Yiu (Tabelas 4.6 e 4.7, pp. 93–94). De que maneira a *flag C* (*carry*) se comporta em uma operação de subtração (**SUBS**) no ARM? Por que razão ela funciona como “NOT *borrow*”, assumindo o valor 1 justamente quando **não** houve empréstimo? Compare esse comportamento com o da *flag* de *carry* em uma subtração no x86, citando explicitamente a fonte consultada.
8. Consulte as Seções 4.2.3 (bit T do EPSR) e 5.6.9 (*Program flow control*) do livro do Yiu. Por que razão todo endereço de função utilizado como destino de um desvio com troca de estado (**BX**, **BLX**), ou armazenado na tabela de vetores, precisa necessariamente ter o bit 0 igual a 1? O que ocorre no Cortex-M caso o programa desvie para um endereço cujo bit 0 seja igual a 0?

Ponte para o Próximo Capítulo

9. Segundo a arquitetura Cortex-M, ao sair do estado de reset, o processador lê o valor inicial do *stack pointer* no endereço `0x00000000` e o endereço do tratador de reset em `0x00000004`. No STM32F411, contudo, a memória Flash, onde o programa é efetivamente gravado, tem início em `0x08000000`. Pesquise de que maneira o microcontrolador resolve essa aparente contradição, e o que vêm a ser os pinos **B00T0** e **B00T1** da Blackpill.